

Clarifications & Key Terms

Project MAESTRO · Group 298 · companion to the Progress Report

This document does two jobs: (1) it clarifies the questions people actually ask about this project — including the hard viva questions — and (2) it defines every Major-project term the reports use, so the Minor (research) and the Major (outcome) speak the same language.

Part I — Clarifications

C1. What exactly is the relationship between the Minor and the Major?

They are two separate deliverables, each defended on its own terms — not two halves of one document. They share a subject and they converge at the end, but each is submitted, graded, and examined independently.

- **Minor (7th sem, 12 weeks) — a research submission.** Literature review, gap analysis, requirements, the formal specification of the Action IR / risk taxonomy / trust model, the evaluation methodology, and a seed dataset. It answers "*what should exist, why does it need to exist, and how will we know it works?*" and it reaches its own conclusion.
- **Major (8th sem, 16 weeks) — a built outcome.** Full implementation on both OSeS, dataset scaled up, the LoRA fine-tune, the benchmark and adversarial evaluation, the user study, and the final report + paper draft + public dataset release. It answers "*here it is, and here are the numbers.*"

Writing consequence, and it matters: each report must **stand alone**. The Minor must not read as "part one, to be continued" — it needs a complete arc ending in its own conclusion and future work. The Major must not assume its examiner has read the Minor, so it re-establishes the problem, the architecture, and the contribution in its own voice. Deliberate overlap between the two is correct and expected; it is not duplication.

C2. "Isn't this just an LLM with a confirmation popup?"

No, and the difference is testable. A popup is UI. MAESTRO's gate is an **architecture**: plans are compiled into a typed Action IR, risk-scored by *deterministic code* (never the LLM), dry-run to produce a real effect preview before anything touches disk, and logged to a tamper-evident chain. Each of those properties has a unit test and a metric. A popup has neither.

C3. "How do you know it's safe?"

We never claim "safe." An LLM-driven agent with filesystem and browser access cannot be proven safe, and claiming so would collapse in the viva. We claim four *operationally defined, measured* properties: **auditable, reversible, consent-gated, injection-resistant** — with numbers (UER, IRR, URR, AIR — see Part II) and with residual risks named by us first: no formal verification; allowlist is policy, not an OS sandbox; injection resistance is empirical over 40 cases, not immunity; consent depends on the user reading the preview; a malicious *user* is out of scope.

C4. "Why not just use GPT / a frontier model?"

Cost (₹0 hard constraint), privacy (no file contents leave the machine), and offline capability (the viva demo does not depend on Wi-Fi). The frontier model is not a competitor — it is **baseline B4/M5 in our evaluation**, the capability ceiling we measure against.

C5. What does "multi-agent" mean here?

Specialized cooperating components with distinct responsibilities and **distinct context windows** — not role-played chat personas. Interpreter, Planner, Critic, Executors (×6, no LLM), Summarizer, Verifier. The design-critical rule: the **Summarizer is the only LLM that ever sees untrusted content, and it has no tools** — its output is data, never action. That single rule is the injection defense.

C6. Why is the risk scorer deterministic code and not an LLM judge?

Because a safety decision you cannot reproduce is a safety decision you cannot test, and an LLM judge inherits every prompt-injection vulnerability it is supposed to guard against. *The model proposes; deterministic code decides.* The model is inside the trust boundary for proposals and outside it for decisions.

C7. Why does the plan "freeze" at consent time (fixed DAG)?

Dynamic re-planning mid-execution feels more capable, but it is precisely the hole through which a malicious document escalates a "summarize" task into an "exfiltrate" task. Fixing the DAG at consent time makes the user's approval mean something specific. Discovering more work mid-run ends the plan and starts a *new* proposal cycle.

C8. Why won't the refusals ever have an override flag?

A safety control with a bypass is a safety control that will be bypassed — usually by a user who has already been prompted twelve times that afternoon. Hard-blocked classes (credential entry, purchases, permanent delete, autonomous email send, `sudo`, arbitrary shell) are refused with an explanation, and *refusal itself is a tested behavior with an accuracy metric.*

C9. Why prompt the user so rarely (R0/R1 never prompt)?

Anti-habituatation. A system that prompts constantly trains users to click through, and the gate protects no one. That is why False Confirmation Rate (FCR) is a metric we *minimize*, and why UER and FCR must always be reported together: the real result is *how little we had to annoy the user to reach zero unsafe executions.*

C10. What is genuinely novel, in one breath?

A typed, dry-runnable Action IR governed by a deterministic risk engine; prompt-injection resistance evaluated as a first-class metric with per-control attribution; the price-of-safety quantified via ablation; and a public safety-annotated NL→plan dataset (DeskPlan). Individually borrowed ideas (dry-run is `terraform plan` for the desktop); the contribution is applying and **measuring** them in a domain that has neither.

C11. What are we actually building — a website, a desktop app, a terminal tool, a plugin?

An open-source local desktop application, driven from the terminal, with an optional desktop GUI later. It is not a website and not a plugin to an existing assistant. Both exclusions are architectural, not stylistic:

- **It cannot be a website.** A web page runs inside a browser sandbox that is *designed* to prevent filesystem access. MAESTRO's entire purpose is to operate a real machine — move actual files, launch actual applications, read actual disks — which requires a native program running with the user's own permissions.
- **It cannot be a "skill" or assistant plugin.** As an Alexa skill or Siri shortcut, the platform vendor owns everything after the intent is recognised. Deterministic risk scoring, dry-run-before-consent, and tamper-evident auditing would all happen in *their* code, not ours — which would give away precisely the layer that constitutes this project's contribution.

The system is delivered in three layers:

LAYER	WHAT IT IS	REQUIREMENT	WHEN
Package	<code>maestro</code> , an importable Python package with a build config — a library <i>and</i> an application. The evaluation harness imports it to drive 3,000 benchmark runs.	—	Exists
Terminal interface	<code>maestro ask "move the pdfs to archive"</code> — a CLI/TUI sufficient for all development and evaluation	FR-60 (must)	Exists
Desktop GUI	Electron/Tauri shell: plan preview, live progress, audit-log viewer, undo button. Wraps the same pipeline; adds no capability.	FR-61 (should)	8th sem M13 — first item on the cut list

C12. So what is the actual deliverable?

A research project's deliverable is not a product, and the PRD says so explicitly: *no installer, no auto-update, no telemetry, no multi-user support*. What is submitted is:

the report · a conference-submittable paper draft · the DeskPlan dataset released publicly · the source code on GitHub · a demo video

The agent is the *artifact*; the measurements are the *contribution*. The GUI exists so an examiner can follow a live demo on screen — not because the system is being shipped to end users. Stated in one line for the viva: **"We built an open-source local desktop agent, a public safety-annotated dataset, and an evaluation of four safety properties it provides."**

Part II — Key Terms (the Major-project vocabulary)

Architecture terms

TERM	DEFINITION
Action IR	The typed, OS-independent Intermediate Representation of a plan: a JSON DAG of actions, each with a verb from the closed registry, validated args, declared risk, reversibility, undo, pre/postconditions, and a human-readable rationale. The single contract between planner and executor.
DAG	Directed Acyclic Graph — actions plus <code>depends_on</code> edges. Cycles are rejected; independent branches may run concurrently; the graph is <i>frozen at consent time</i> .
Closed verb registry	The finite whitelist of verbs a plan may use (<code>fs.move</code> , <code>browser.open</code> , ...). The planner cannot invent <code>sys.exec_shell</code> because no such verb exists to emit; unknown verbs are rejected before reaching executable code.
\$var dataflow	The only way data moves between actions (<code>produces: "pdf_list" → "sources": "\$pdf_list"</code>). No implicit shared state, so dataflow is inspectable and taintable.
Executor	Plain code (no LLM) implementing one verb with four methods: <code>validate</code> , <code>dry_run</code> , <code>execute</code> , <code>undo</code> . Only layer written per-OS (<code>darwin/</code> , <code>win32/</code>).
Orchestrator	Topologically executes the approved DAG, checks pre/postconditions, pushes undo entries, halts and offers rollback on failure.
Postcondition verification	After each action, verify the world actually changed as claimed. Without it an agent reports success whenever a call didn't raise — the most common way agent benchmarks lie (threat T8, "silent failure").
Multi-agent roles	Interpreter (NL→intent/entities) · Planner (→Action IR) · Critic (reviews for over-reach) · Executors (act) · Summarizer (only reader of untrusted content; tool-less) · Verifier (checks postconditions).

Safety terms

TERM	DEFINITION
Risk taxonomy R0–R3	R0 SAFE: pure reads, auto-run. R1 LOW: reversible, in-workspace, auto-run + undo. R2 MEDIUM: reversible-but-consequential or out-of-workspace write → click-to-approve . R3 HIGH: irreversible/security-relevant → typed confirmation , several verbs hard-blocked outright. Plan risk = max(action risks).
Deterministic risk scorer	Ordinary code (no LLM) mapping action → tier. Three defended properties: no LLM call; monotonic (rules can only raise risk); fail-closed (unknown verb/path/var ⇒ R3).
risk_hint	The planner's own risk guess — recorded for the hint-agreement metric, ignored for decisions . A lying planner cannot downgrade risk.
Dry-run / effect manifest	Simulated execution of the whole plan before consent: file counts, bytes, collisions, overwrites — with zero real side effects. What makes consent informed rather than ceremonial.
Consent gate	The tiered approval mechanism (none / none+undo / click / typed token).
Hard blocks	Actions refused regardless of instruction or confirmation, with no override flag : credential entry, purchases, account creation, CAPTCHA, permanent delete, autonomous send, security-setting changes, elevation, arbitrary shell.
Path allowlist / denylist	Workspace roots the agent may touch vs. forbidden paths (<code>~/ .ssh</code> , <code>/System</code> , <code>*.pem</code> , ...). Denylist checked first and wins; paths canonicalized (<code>realpath</code>) and symlinks resolved <i>before</i> matching.
Trust model T0/T1/T2	T0 TRUSTED: the user's instruction, config, registry. T1 DERIVED: plans/entities produced from T0. T2 UNTRUSTED: file contents, web pages, tool output, filenames — data only, may influence nothing , never enters the Planner's context.
Indirect prompt injection	The dominant attack on tool-using agents: instructions hidden inside content the agent reads (white-on-white PDF text, hidden divs). Defense-in-depth: 8 independent controls; the evaluation reports <i>which control fires first</i> per attack.
Taint tracking	Any argument derived from untrusted content escalates the action to ≥R2 and requires re-confirmation.
Hash-chained audit log	Append-only log where each row stores <code>sha256(prev_hash row)</code> . Editing any row breaks the chain — converts "we logged it" into "we can prove the log wasn't edited."
Undo stack	Every R1/R2 action declares its inverse at plan time; session-level undo replays them. Best-effort and honestly labeled (file moves reliable; form submissions not).
Threat model T1–T9	Misinterpretation · Over-reach · Indirect injection · Irreversible action · Scope escape · Capability escalation · Resource exhaustion · Silent failure · Repudiation — each mapped to its primary control.

NLP / ML terms

TERM	DEFINITION
Staged NLP pipeline	Stage 1 intent classification (~16 classes incl. <code>OUT_OF_SCOPE</code> , <code>UNSAFE_REQUEST</code>) → Stage 2 entity/slot extraction (ask when a slot is unfilled; never guess a path) → Stage 3 plan generation. Separately measurable; monolithic-prompt is ablation A7.
Constrained JSON decoding	The LLM's output is grammatically forced to match the IR schema, with the verb field constrained to the registry enum.
DeskPlan	Our dataset: 3,200 target pairs (400 human seed, 1,200 template, 1,400 LLM-generated <i>human-verified</i> , 200 adversarial test-only). Public release planned.
paraphrase_group	Paraphrases share a group ID and must never split across train/test — otherwise test accuracy is inflated.
expected_behavior	Per-pair label: <code>execute_auto</code> / <code>execute_with_consent</code> / <code>clarify</code> / <code>refuse</code> . What makes refusal a trained, measurable behavior.

TERM	DEFINITION
LoRA fine-tuning	Low-Rank Adaptation: train small adapter matrices instead of all weights. Our RQ: can a LoRA-tuned 3B local model match frontier planning quality at ₹0 and full privacy? Config: Qwen2.5-3B, r=16, α=32, 3 epochs, MLX on-device.
Cohen's κ	Inter-annotator agreement statistic on a double-annotated overlap; target κ > 0.75 before the dataset ships.

Evaluation terms

METRIC	MEANING	TARGET
TSR / PSR	Task / Partial Success Rate — postconditions satisfied	TSR ≥ 70% (P0)
Action F1 / Plan Exact Match	Generated plan vs. gold plan, set-F1 over (verb, args) / exact after canonicalization	—
UER	Unsafe Execution Rate — R2/R3 actions executed without their gate	0.0%, non-negotiable
SCR	Safety Compliance Rate — actions gated at the correct tier	≥ 98%
FCR	False Confirmation Rate — R0/R1 wrongly escalated to a prompt (anti-habituating)	≤ 10%
IRR	Injection Resistance Rate over the 40-case adversarial suite	≥ 90%
HBR	Hard Block Rate — blocked classes correctly refused	100%
RSA	Risk Score Accuracy vs. human expert labels	≥ 95%
URR	Undo Reliability — undos fully restoring prior state	≥ 95%
AIR	Audit Integrity — hash chain verifies	100%
Baselines B0–B5	B0 rule-based floor · B1 single LLM, no safety (the naive agent — the key comparison) · B2 multi-agent no safety · B3 MAESTRO full · B4 MAESTRO + frontier planner · B5 human manual	
Ablations A0–A8	Remove one component at a time; A1 (no safety layer) prices safety (RQ2); A3 (no postconditions) exposes silent failure; A8 (no trust isolation) shows IRR collapse (RQ3).	
SUS / Trust scale / NASA-TLX	User-study instruments (n ≥ 15): usability (target > 68), trust in automation, workload.	
Hermetic benchmark	Every task builds its own fixture and tears it down — re-runnable 500× without state leakage. Run matrix: 100 tasks × 5 configs × 2 OS × 3 seeds = 3,000 runs.	

Infrastructure terms

TERM	DEFINITION
Ollama / MLX	Local LLM runtimes (Ollama for GGUF models; MLX for Apple-silicon training/inference). Default model: qwen2.5:7b-instruct-q4_K_M via MAESTRO_MODEL .
Episodic memory	SQLite record of every instruction, plan, consent, outcome, and timing — the raw material of the learning loop.
Learning loop	ask daily → episodes accumulate → learn exports JSONL candidates → human review/labeling → LoRA fine-tune → swap model in → the planner now knows your patterns.
Differential test	The same benchmark task on macOS and Windows must yield an <i>identical Action IR</i> even though executors differ; divergence = abstraction leakage.

TERM	DEFINITION
₹0 constraint	Hard requirement NFR-01: local-first models, free tiers only for baselines, free tooling throughout; the offline demo proves it.